

Nesta etapa da pesquisa, foram utilizadas as seguintes bibliotecas *Python* para o processamento e organização dos dados brutos:

- **Pandas (pandas)**: Biblioteca fundamental para manipulação e análise de dados. Foi utilizada para carregar o dataset, realizar transformações, filtrar dados e criar novas colunas.
- **NumPy (numpy)**: Utilizada para operações numéricas eficientes, como cálculos matemáticos e manipulação de arrays. Foi empregada em operações estatísticas e para lidar com valores nulos ou inválidos.
- **Re (re)**: Biblioteca nativa para buscas, substituições e divisões de texto com expressões regulares básicas (como URLs, menções, hashtags e emojis).
- **Emoji (emoji)**: Biblioteca para manipulação de emojis em textos. Foi usada para remover emojis dos tweets e links, garantindo a padronização dos dados.
- **Hashlib (hashlib)**: Usada para gerar pseudônimos únicos por meio de funções de hash (SHA-256), garantindo a anonimização dos dados de usuários e tweets.
- **Regex (regex)**: Biblioteca compatível com a sintaxe de **re**, mas traz recursos avançados: lookbehind de tamanho variável, propriedades Unicode completas (`\p{Letter}`), correspondência fuzzy, divisões condicionais e controle de backtracking. Útil quando padrões complexos ou suporte total a Unicode são necessários.
- **DiffLib (difflib)**: Utilizada para comparar textos e encontrar similaridades, ajudando a identificar tweets truncados e seus originais.
- **Swifter (swifter)**: Biblioteca que acelera operações em **DataFrames** do Pandas, aplicando paralelismo. Foi usada para otimizar o processamento de funções em grandes volumes de dados.
- **Requests (requests)**: Utilizada para fazer requisições HTTP, principalmente na expansão de URLs encurtadas, seguindo redirecionamentos para obter os links originais.
- **Tqdm (tqdm)**: Adiciona barras de progresso para loops e operações demoradas, melhorando a visualização do andamento do processamento, como na expansão de URLs.

```
[1]: !pip install emoji -q
      !pip install swifter -q
```

```
[2]: import numpy                as np
      import pandas              as pd
      import re
      import emoji
      import hashlib
      import regex               as reg
      from difflib import SequenceMatcher
      import swifter
      import requests
      from requests.exceptions import RequestException
```

```
from tqdm import tqdm
```

```
[3]: # Carreagando dataset  
df = pd.read_csv(  
    '/content/drive/MyDrive/TCC/Data/TUEBR22_bruto.csv',  
    sep="," ,  
    low_memory=False)
```

B.1 Limpeza e Padronização de Dados

```
[3]: # Vizualizando estrutura do dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 142425 entries, 0 to 142424
Data columns (total 20 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   tweet_id              142425 non-null object
 1   tweet                 142395 non-null object
 2   link                  18876 non-null  object
 3   created_at            142424 non-null object
 4   in_reply_to_status_id 28374 non-null  object
 5   in_reply_to_user_id   28578 non-null  object
 6   retweet_count          142424 non-null float64
 7   favorite_count         142425 non-null int64
 8   user_id                142425 non-null object
 9   name                   142417 non-null object
10   screen_name            142425 non-null object
11   location               76114 non-null  object
12   followers_count        142425 non-null int64
13   created_at.1           142425 non-null object
14   verified                142425 non-null bool
15   dt_coleta              142425 non-null object
16   geo                     2 non-null      object
17   coordinates            2 non-null      object
18   place                  1062 non-null   object
19   lang                   137929 non-null object
dtypes: bool(1), float64(1), int64(2), object(16)
memory usage: 20.8+ MB
```

```
[4]: # Renomeando features
df = df.rename(columns={
    'tweet': 'raw_tweet',
    'created_at': 'tweet_created_at',
    'in_reply_to_status_id': 'reply_to_tweet_id',
    'in_reply_to_user_id': 'reply_to_user_id',
    'created_at.1': 'user_created_at',
    'screen_name': 'username',
    'dt_coleta': 'collection_date',
    'retweet_count': 'retweets',
    'favorite_count': 'likes',
    'verified': 'is_verified'
})
```

```
[5]: df.columns.tolist() # Validando alteração
```

```
[5]: ['tweet_id',
      'raw_tweet',
      'link',
      'tweet_created_at',
      'reply_to_tweet_id',
      'reply_to_user_id',
      'retweets',
      'likes',
      'user_id',
      'name',
      'username',
      'location',
      'followers_count',
      'user_created_at',
      'is_verified',
      'collection_date',
      'geo',
      'coordinates',
      'place',
      'lang']
```

```
[6]: # Verificando metadados
df[[
    'location',
    'geo',
    'coordinates',
    'place',
    'lang'
]].sample(10)
```

```
[6]:
```

	location	geo	coordinates	place	lang
77360	Salvador, Brasil	NaN	NaN	NaN	pt
182	Brasil	NaN	NaN	NaN	NaN
7792	NaN	NaN	NaN	NaN	pt
47047	Japão	NaN	NaN	NaN	pt
105776	NaN	NaN	NaN	NaN	pt
97852	Estados Unidos	NaN	NaN	NaN	pt
74009	Araguari, Brasil	NaN	NaN	NaN	pt
79882	NaN	NaN	NaN	NaN	pt
125406	Ceará, Brasil	NaN	NaN	NaN	pt
138132	NaN	NaN	NaN	NaN	pt

```
[7]: """Removendo metadados com muitos valores nulos
      ou com dados irrelevantes para a pesquisa
      """
cols_remove = [
    'location',
```

```

        'geo',
        'coordinates',
        'place',
        'lang'
    ]
    df.drop(columns=cols_remove, inplace=True)

```

```

[8]: # Convertendo para o formato Datetime
df['tweet_created_at'] = pd.to_datetime(df['tweet_created_at'])
df['user_created_at'] = pd.to_datetime(df['user_created_at'])
df['collection_date'] = pd.to_datetime(df['collection_date'])

```

```

[9]: # Tweets sem data de criação
df[df['tweet_created_at'].isna()]['tweet_created_at']

```

```

[9]: 33983    NaT
      Name: tweet_created_at, dtype: datetime64[ns]

```

```

[10]: # Removendo tweets sem data de criação
df.drop(df[df['tweet_created_at'].isna()].index, inplace=True)
df.reset_index(drop=True, inplace=True)

```

```

[11]: # Convertendo para o formato int
df['retweets'] = pd.to_numeric(df['retweets'], errors='coerce').astype('Int64')

```

```

[12]: # Filtrando linhas onde a coluna 'raw_tweet' está vazia (string vazia ou NaN)
tweets_vazios = df.loc[
    (df['raw_tweet'].isna()) | # Verifica valores NaN
    (df['raw_tweet'].str.strip() == '') # Verifica strings vazias ou com espaços
]

# Quantitativo de tweets vazios
len(tweets_vazios)

```

```

[12]: 30

```

```

[13]: """
      Verificando dados dos tweets vazios (ao analisar os links, observa-se que eles
      direcionam para notícias relacionadas a urna, sendo assim a falta do conteúdo
      do tweet deve ser por conta de falha na escrita)
      """
      tweets_vazios['raw_tweet'].sample(10)

```

```

[13]: 1585    NaN
      2043    NaN
      1598    NaN
      3768    NaN
      2075    NaN

```

```
4      NaN
3238   NaN
2288   NaN
1988   NaN
1876   NaN
Name: raw_tweet, dtype: object
```

```
[14]: # Excluindo tweets vazios
condicao_vazios = df['raw_tweet'].isna() | (df['raw_tweet'].str.strip() == '')
df.drop(df[condicao_vazios].index, inplace=True)
df.reset_index(drop=True, inplace=True)
```

```
[15]: # Excluindo tweets muito curtos e sem link
df = df[~((df['link'].isna()) & (df['raw_tweet'].str.len() < 10))]
df = df.reset_index(drop=True)
```

```
[16]: # Contar duplicatas na coluna tweet_id (chave única)
df.duplicated(subset=['tweet_id'], keep=False).sum()
```

```
[16]: np.int64(6)
```

```
[17]: # Filtrar linhas duplicadas e ordenar (Erro na escrita, trucando o ID do tweet)
duplicates_df = df[df.duplicated(subset=['tweet_id'], keep=False)]\
                .sort_values(by='tweet_id', ascending=False)

# Mostrar colunas relevantes
duplicates_df[['tweet_id', 'collection_date']]
```

```
[17]:      tweet_id      collection_date
87163  1,58631E+18  2022-10-29 07:43:55
87198  1,58631E+18  2022-10-29 08:09:48
50637  1,57893E+18  2022-10-08 22:50:35
50660  1,57893E+18  2022-10-08 23:04:53
22507  1,57449E+18  2022-09-26 17:35:10
22537  1,57449E+18  2022-09-26 17:53:50
```

```
[18]: # Excluindo tweets duplicados
df.drop_duplicates(subset='tweet_id', inplace=True)
df.reset_index(drop=True, inplace=True)
```

```
[19]: # Contagem de contas verificadas ou não
df['is_verified'].value_counts()
```

```
[19]: is_verified
False    141680
True       710
Name: count, dtype: int64
```

```
[20]: # Converte True/False para 1/0
df['is_verified'] = df['is_verified'].astype(int)
```

```
[21]: # Validando alteração
df['is_verified'].value_counts()
```

```
[21]: is_verified
0      141680
1         710
Name: count, dtype: int64
```

```
[22]: # Tweets sem nome do usuário, serão mantidos visto que não serão utilizados
df[df['name'].isna()][['name']]
```

```
[22]: 7134      NaN
13884      NaN
14184      NaN
35038      NaN
76198      NaN
83231      NaN
114876     NaN
137169     NaN
Name: name, dtype: object
```

```
[23]: # Define a máscara para identificar valores a substituir
mask = df['link'].str.len().fillna(0) < 23
# Substitui por NaN onde a máscara é True
df.loc[mask, 'link'] = np.nan
```

```
[24]: # Filtra URLs com reticência ou pontos finais (concatenado)
df[df['link'].str.contains('...|\.{3}', na=False)][['link']]
```

```
[24]:
```

	link
2508	https://t.co/gFPAKICqtd...
14124	https://t.co/zbSTAWOPA6...
14132	https://t.co/zbSTAWOPA6...
14147	https://t.co/zbSTAWOPA6...
14173	https://t.co/zbSTAWOPA6...
14223	https://t.co/zbSTAWOPA6...
14233	https://t.co/zbSTAWOPA6...
14348	https://t.co/zbSTAWOPA6...
15701	https://t.co/zbSTAWOPA6...
43417	https://t.co/CnzSYUAY59...
43424	https://t.co/CnzSYUAY59...
43460	https://t.co/CnzSYUAY59...
43593	https://t.co/CnzSYUAY59...
43693	https://t.co/CnzSYUAY59...
44085	https://t.co/CnzSYUAY59...

```
44457          https://t.co/CnzSYUAY59...
74038          https://t.co/sSm2vwgxjN...
90751          https://t.co/dgJpfVaoP...
92414  https://g1.globo.com/fato-ou-fake/eleicoes/not...
93820          https://t.co/OU1S5rvwdT...
135239         https://t.co/Ko1RFMN8u8...
```

```
[25]: # URL truncada que não esta encurtada
df.iloc[92414]['link']
```

```
[25]: 'https://g1.globo.com/fato-ou-fake/eleicoes/notici...'
```

```
[26]: # Substitui por NaN
df.loc[92414, 'link'] = np.nan
df.iloc[92414]['link']
```

```
[26]: nan
```

```
[27]: # Função que aplica a lógica de limpeza e validação
def limpar_validar_url(url):
    if not isinstance(url, str):
        return None # ignora valores nulos ou não string

    # Remove reticência e pontos finais
    if url.endswith('...'):
        url = url[:-1]
    elif url.endswith('...'):
        url = url[:-3]

    clean_ = url

    # Verifica comprimento e regex
    if len(clean_) >= 23 and re.match(r'^https?:\/\/[^\s]+\.[^\s]+$', clean_):
        return clean_
    else:
        return None # URL inválida

# Aplica à coluna 'link'
df['link'] = df['link'].apply(limpar_validar_url)
```

```
[28]: # Verificando alteração
df[df['link'].str.contains('...|\.{3}', na=False)][['link']]
```

```
[28]: Empty DataFrame
      Columns: [link]
      Index: []
```

```
[29]: # Removendo emojis que foram coletados por engano junto ao link
# Observado apenas um caso: 258
df['link'] = df['link'].apply(
    lambda x: emoji.replace_emoji(x, '') if pd.notnull(x) else x
)
```

```
[30]: # Links com ruídos
df[df['link'].str.contains(r'[\.,)\]\}\$'], na=False)][ 'link']
```

```
[30]: 1822      https://t.co/3IiZXAgwd0.
2637      https://t.co/bdeFrDjXmU)
4454      https://t.co/f3w7mIU6Wo.
10560     https://t.co/QgI2Oher3a.
24626     https://t.co/V03hWyqCe3,
...
127635    https://t.co/x7sibHWSb1)
128847    https://t.co/x7sibHWSb1)
129276    https://t.co/9oTYHdGPYj,
130409    https://t.co/QY15nmCYnq.
131105    https://t.co/vwCg5fhqN4.
Name: link, Length: 85, dtype: object
```

```
[31]: # Removendo ruídos (., ), ,, ]) no final dos link
# Observado em alguns caso: 1822, 2637, 24626...
df['link'] = df['link'].str.extract(r'(https://t\.co/[a-zA-Z0-9]+)')
```

```
[32]: # Substitui strings vazias ou espaços por NaN
# Observado em apenas um caso: 19695
df['link'] = df['link'].replace(r'^\s*$', np.nan, regex=True)
```

```
[33]: df[df['link'].isna()][ 'link'].sample(10) # Verificando
```

```
[33]: 36857      NaN
115317      NaN
106375      NaN
956         NaN
68801       NaN
65329       NaN
77901       NaN
137115      NaN
122865      NaN
15186       NaN
Name: link, dtype: object
```

```
[34]: df.info() # Validando alterações

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 142390 entries, 0 to 142389
```

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	tweet_id	142390 non-null	object
1	raw_tweet	142390 non-null	object
2	link	17049 non-null	object
3	tweet_created_at	142390 non-null	datetime64[ns]
4	reply_to_tweet_id	28370 non-null	object
5	reply_to_user_id	28575 non-null	object
6	retweets	142390 non-null	Int64
7	likes	142390 non-null	int64
8	user_id	142390 non-null	object
9	name	142382 non-null	object
10	username	142390 non-null	object
11	followers_count	142390 non-null	int64
12	user_created_at	142390 non-null	datetime64[ns]
13	is_verified	142390 non-null	int64
14	collection_date	142390 non-null	datetime64[ns]

dtypes: Int64(1), datetime64[ns](3), int64(3), object(8)

memory usage: 16.4+ MB

B.2 Pseudonimização

```
[35]: # ===== Configurações =====
SALT = "#####"

# ===== Dicionários de Mapeamento =====
user_mapping = {}          # {(user_id, username): (pseudo_id, name)}
username_to_user_id = {}   # Mapeamento reverso {username: user_id}
tweet_mapping = {}         # {tweet_id: pseudo_id}
external_ids = set()       # {(tipo, id_original)}

# ===== Funções Base =====
def generate_pseudo_id(original_id, prefix=""):
    return f"{prefix}_" + hashlib.sha256((str(original_id) + SALT).encode()
                                           ).hexdigest()[:12].upper()

# ===== Pseudonimização de Usuários =====
def pseudonymize_user(row):
    # Processa cada usuário do DataFrame e atualiza mapeamentos
    user_id = row["user_id"]
    username = row["username"].strip('@').lower() # Remove @ existente
    name = row["name"]

    # Atualiza mapeamento reverso
    if username not in username_to_user_id:
        username_to_user_id[username] = user_id

    # Cria entrada no mapeamento principal
    user_key = (user_id, username)
    if user_key not in user_mapping:
        pseudo = generate_pseudo_id(user_id, prefix="@USER")
        user_mapping[user_key] = (pseudo, name)

    return user_mapping[user_key][0]

# ===== Pseudonimização de Tweets =====
def pseudonymize_tweet(tweet_id):
    # Gera ID pseudônimo para tweets
    if pd.isna(tweet_id) or tweet_id == "":
        return np.nan
    if tweet_id not in tweet_mapping:
        tweet_mapping[tweet_id] = generate_pseudo_id(tweet_id, prefix="TWEET")
    return tweet_mapping[tweet_id]

def pseudonymize_mentions(text):
    # Substitui menções @username por pseudônimos mantendo um único @
    if pd.isna(text):
```

```

        return text

    # Extrai menção
    mentions = re.findall(r'(?<!\w)@\w{1,25}(?!\\w)', text)

    for username in mentions:
        clean_username = username.strip('@').lower()
        # Usuários do dataset
        if clean_username in username_to_user_id:
            user_id = username_to_user_id[clean_username]
            pseudo = user_mapping[(user_id, clean_username)][0]
            # Usuários externos
        else:
            user_key = (None, clean_username)
            if user_key not in user_mapping:
                pseudo_ext = generate_pseudo_id(clean_username,
                                                  prefix="@EXT_USER")
                user_mapping[user_key] = (pseudo_ext, None)
                external_ids.add(("user", clean_username))
            pseudo = user_mapping[user_key][0]

        # Substitui a menção completa (@username)
        pattern = rf'(?<!\w)@{re.escape(clean_username)}(?!\\w_)'\
        text, count = re.subn(pattern, pseudo, text, flags=re.IGNORECASE)

    return text

# ===== Processamento Principal =====

# Pseudonimiza usuários principais
df["user_pseudo"] = df.apply(pseudonymize_user, axis=1)

# Pseudonimiza tweets
df["tweet_id_pseudo"] = df["tweet_id"].apply(pseudonymize_tweet)

# Processa texto dos tweets
df["tweet_pseudo"] = df["raw_tweet"].apply(pseudonymize_mentions)

# Pseudonimiza replies
def process_reply_user(id_):
    if pd.isna(id_) or id_ == "":
        return np.nan
    if id_ in df["user_id"].values:
        return generate_pseudo_id(id_, prefix="@USER")
    else:
        external_ids.add(("reply_user", id_))
        return generate_pseudo_id(id_, prefix="@EXT_USER")

```

```

df["reply_to_user_pseudo"] = df["reply_to_user_id"].apply(process_reply_user)

def process_reply_tweet(id_):
    if pd.isna(id_) or id_ == "":
        return np.nan
    if id_ in tweet_mapping:
        return tweet_mapping[id_]
    else:
        external_ids.add(("reply_tweet", id_))
        return generate_pseudo_id(id_, prefix="EXT_TWEET")

df["reply_to_tweet_pseudo"] = df["reply_to_tweet_id"].apply(process_reply_tweet)

# ===== Exportação de Mapeamentos =====
# Mapeamento de usuários (incluindo externos)
users_data = []
for (user_id, username), (pseudo, name) in user_mapping.items():
    users_data.append({
        "user_id_original": user_id if user_id is not None else np.nan,
        "username_original": f"@{username}",
        "user_pseudo": pseudo,
        "name_original": name if name is not None else np.nan
    })
pd.DataFrame(users_data)
).to_csv("/content/drive/MyDrive/TCC/Data/mapeamento_usuarios.csv", index=False)

# Mapeamento de tweets
pd.DataFrame(
    list(tweet_mapping.items()),
    columns=["tweet_id_original", "tweet_id_pseudo"]
).to_csv("/content/drive/MyDrive/TCC/Data/mapeamento_tweets.csv", index=False)

# IDs externos
if external_ids:
    pd.DataFrame(
        list(external_ids),
        columns=["tipo", "id_original"]
    ).to_csv("/content/drive/MyDrive/TCC/Data/ids_externos.csv", index=False)

```

Validando Pseudonimização:

```

[36]: # Carregar os arquivos gerados
df_original = df.copy() # Substitua pelo seu arquivo
df_map_users = pd.read_csv(
    "/content/drive/MyDrive/TCC/Data/mapeamento_usuarios.csv")
df_map_tweets = pd.read_csv(

```

```

        "/content/drive/MyDrive/TCC/Data/mapeamento_tweets.csv")

# ===== Validação de Usuários =====
print("\n=== Validação de Usuários ===")

# Contagem de usuários originais vs pseudônimos
original_users_count = df_original["user_id"].nunique()
pseudo_users_count = df_map_users["user_pseudo"].nunique()

print(f"Usuários originais únicos: {original_users_count}")
print(f"Pseudônimos de usuários únicos: {pseudo_users_count}")

# ===== Validação de Tweets =====
print("\n=== Validação de Tweets ===")

# Contagem de tweets originais vs pseudônimos
original_tweets_count = df_original["tweet_id"].nunique()
pseudo_tweets_count = df_map_tweets["tweet_id_pseudo"].nunique()

print(f"Tweets originais únicos: {original_tweets_count}")
print(f"Pseudônimos de tweets únicos: {pseudo_tweets_count}")

# ===== Validação de Menções e Replies =====
print("\n=== Validação de Menções/Replies ===")

# Menções externas nos textos
external_mentions_in_tweets = df_map_users[
    (df_map_users["user_pseudo"].str.contains("@EXT_USER")) &
    (df_map_users["user_id_original"].isnull())
]["username_original"].nunique()

print(f"Menções a usuários externos em textos: {external_mentions_in_tweets}")

# Replies externos
try:
    df_ext = pd.read_csv("/content/drive/MyDrive/TCC/Data/ids_externos.csv")

    ext_user= df_ext[df_ext["tipo"] == "reply_user"]["id_original"].nunique()
    ext_tweets= df_ext[df_ext["tipo"] == "reply_tweet"]["id_original"].nunique()
    print(f"Replies para usuários externos: {ext_user}")
    print(f"Replies para tweets externos: {ext_tweets}")
except FileNotFoundError:
    print("Nenhum reply externo encontrado.")

# ===== Verificação de Duplicidades =====

```

```

print("\n=== Verificação de Duplicidades ===")

# Pseudônimos duplicados (deveria ser zero)
duplicate_pseudo_users = df_map_users["user_pseudo"].duplicated().sum()
print(f"Pseudônimos de usuários duplicados: {duplicate_pseudo_users}")

duplicate_pseudo_tweets = df_map_tweets["tweet_id_pseudo"].duplicated().sum()
print(f"Pseudônimos de tweets duplicados: {duplicate_pseudo_tweets}")

```

```

=== Validação de Usuários ===
Usuários originais únicos: 100867
Pseudônimos de usuários únicos: 121513

```

```

=== Validação de Tweets ===
Tweets originais únicos: 142390
Pseudônimos de tweets únicos: 142390

```

```

=== Validação de Menções/Replies ===
Menções a usuários externos em textos: 20646
Replies para usuários externos: 10146
Replies para tweets externos: 23743

```

```

=== Verificação de Duplicidades ===
Pseudônimos de usuários duplicados: 269
Pseudônimos de tweets duplicados: 0

```

Verificando usuários duplicados:

```

[37]: # Filtra e ordena as linhas com user_pseudo duplicado
dupl_order = df_map_users[
    df_map_users["user_pseudo"].duplicated(keep=False)
    ].sort_values(by="user_pseudo")

# Calcula o tamanho de cada grupo e reseta o índice
group = dupl_order.groupby('user_id_original').size().reset_index(name='size')

# Mescla os tamanhos dos grupos de volta ao DataFrame original
duplicados_ordenados = pd.merge(
    dupl_order,
    group, on='user_id_original', how='left')

# Filtra com base na coluna 'size'
result = duplicados_ordenados[duplicados_ordenados['size'] > 1]

result[['user_pseudo', 'size']].sample(10)

```

```
[37]:
```

	user_pseudo	size
277	@USER_89E6E6984CAD	2
506	@USER_F7B57F06B724	2
216	@USER_697DE178A500	2
368	@USER_B599218A1334	3
263	@USER_843D3096655D	2
308	@USER_9637CFD689B0	2
82	@USER_244FF4E80734	2
6	@USER_0574EB23F76B	2
231	@USER_72A69E47149C	2
133	@USER_3F8E5FA60CEE	2

```
[38]: # Veriificando tweets usuários duplicados
USER = '@USER_FFC29CEF459C'
df[df['user_pseudo'].str.contains(USER)][
    ['user_pseudo', 'tweet_id_pseudo', 'tweet_created_at']]
```

```
[38]:
```

	user_pseudo	tweet_id_pseudo	tweet_created_at
70063	@USER_FFC29CEF459C	TWEET_00D8224AD8DC	2022-10-20 00:35:14
121244	@USER_FFC29CEF459C	TWEET_86FF0CCDED2A	2022-11-17 21:27:02

NOTA: Identificou-se que a causa das duplicidades foi a alteração do *username* durante o período de coleta de dados.

Validando Pseudonimização:

```
[39]: # Amostragem de validação
df[['tweet_id_pseudo', 'tweet_pseudo']].sample(10)
```

```
[39]:
```

	tweet_id_pseudo	tweet_pseudo
44676	TWEET_B6E54A91E28C	RT @USER_F0577D5FA2B5: @USER_9C1AC011816D @EXT...
23219	TWEET_2E500710ADDB	RT @EXT_USER_5B29B3164900: Ao apertar 13 na ur...
71323	TWEET_F24BC9F26E2C	RT @USER_5BD39CFDF044: Bolsonaro tá fazendo DE...
81566	TWEET_5D9DE3EF8E05	RT @EXT_USER_9ED1599A439F: SE ISSO AQUI NÃO TE...
131600	TWEET_B0F88180B1C7	RT @EXT_USER_8FC8E6D58A1F: A decisão do TSE nã...
44111	TWEET_B661AC69769F	@USER_C8078A90D9B9 Já ganhou hora q saiu da ca...
78268	TWEET_42117D68E40E	@EXT_USER_03AE82DDA07B Meu aniversário é domin...
75141	TWEET_99636D377A22	faz isso q no dia 30 eu aperto 22 na urna
86423	TWEET_2B4FE8C10F2D	RT @USER_E5148283DBA4: A história é implacável...
57554	TWEET_CDED8A211213	RT @USER_271286EA1893: Em 2018 a Lavajato, na ...

```
[40]: # Selecionado usuário
USER = '@USER_45D2CEC77707'
```

```
[41]: # Validando no dataset de mapeamento de usuários
df_map_users[df_map_users['user_pseudo'].str.contains(USER)]
```

```
[41]:
```

	user_id_original	username_original	user_pseudo	name_original
22901	14594813	@folha	@USER_45D2CEC77707	Folha de S.Paulo

```
[42]: # Validando no dataset original, se está sendo mencionado
df[df['tweet_pseudo'].str.contains(USER)][['tweet_pseudo']].sample(10)
```

```
[42]:                                     tweet_pseudo
63453  @EXT_USER_5322BF39BA91 @EXT_USER_3878A1AE2412 ...
15622  @EXT_USER_B125ED45AD20 @USER_45D2CEC77707 Bot ...
86401  RT @USER_45D2CEC77707: Moraes derruba canal de...
2746   @EXT_USER_B2A5BD29E6CA @USER_45D2CEC77707 @USE...
85416  @USER_45D2CEC77707 Tava pensando em anular, ma...
85203  RT @USER_45D2CEC77707: "Não sei nem se você va...
86213  RT @USER_45D2CEC77707: Moraes derruba canal de...
99149  @USER_45D2CEC77707 E o voto não apareceu na ur...
72823   @USER_45D2CEC77707 600 no bolso e 13 na urna!
141054 @EXT_USER_A22D2A29C5AE @EXT_USER_A163150E3162 ...
```

```
[43]: # Validando no dataset original, se é uma resposta a este usuário
df[df['reply_to_user_pseudo'].str.contains(USER,␣
↳na=False)][['user_pseudo','reply_to_user_pseudo','reply_to_tweet_pseudo']].
↳sample(10)
```

```
[43]:          user_pseudo reply_to_user_pseudo  reply_to_tweet_pseudo
62826  @USER_5947919BB65F  @USER_45D2CEC77707  EXT_TWEET_E40DA5801D2A
139875 @USER_7BE8C07CA04F  @USER_45D2CEC77707  EXT_TWEET_08729E8F797D
4229   @USER_2770B1C48826  @USER_45D2CEC77707  EXT_TWEET_216116E5DB83
44263  @USER_7E7B17228521  @USER_45D2CEC77707  EXT_TWEET_903F9CBB27AD
28573  @USER_8241C5D474BC  @USER_45D2CEC77707  EXT_TWEET_85F130BEBBBF
72002  @USER_7A1E6C5F7A12  @USER_45D2CEC77707  EXT_TWEET_393A3F01D7F0
60692  @USER_E0D4D5E01A9E  @USER_45D2CEC77707  EXT_TWEET_D19CC37FB104
65954  @USER_6150C320C156  @USER_45D2CEC77707  EXT_TWEET_48C506FF67DC
98971  @USER_5835AC3FCEC7  @USER_45D2CEC77707  EXT_TWEET_4F48917718CD
60702  @USER_21E08D64A4D7  @USER_45D2CEC77707  EXT_TWEET_D19CC37FB104
```

```
[44]: # Validando no dataset original, se é o autor do tweet
df[df['user_pseudo'].str.contains(USER)][['user_pseudo','tweet_pseudo']]
```

```
[44]:          user_pseudo                                     tweet_pseudo
27805  @USER_45D2CEC77707  Entenda por que a urna eletrônica brasileira é...
44304  @USER_45D2CEC77707  Folha Informações: É falso que boletim de urna...
93603  @USER_45D2CEC77707  Netanyahu terá maioria frágil e pode voltar ao...
```

```
[45]: # Selecionado tweet
TWEET = 'TWEET_500CBAA5C279'
# Validando no dataset de mapeamento de tweets
df_map_tweets[df_map_tweets['tweet_id_pseudo'].str.contains(TWEET)]
```

```
[45]:          tweet_id_original  tweet_id_pseudo
141865  1598514710445318145  TWEET_500CBAA5C279
```

```
[46]: # Validando no dataset original, se é uma resposta a este tweet
df[df['reply_to_tweet_pseudo'].str.contains(TWEET,
↪na=False)][['tweet_id_pseudo', 'reply_to_tweet_pseudo']]
```

```
[46]: Empty DataFrame
Columns: [tweet_id_pseudo, reply_to_tweet_pseudo]
Index: []
```

```
[47]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 142390 entries, 0 to 142389
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   tweet_id                             142390 non-null  object
1   raw_tweet                           142390 non-null  object
2   link                                 17049 non-null   object
3   tweet_created_at                    142390 non-null  datetime64[ns]
4   reply_to_tweet_id                   28370 non-null   object
5   reply_to_user_id                    28575 non-null   object
6   retweets                            142390 non-null  Int64
7   likes                               142390 non-null  int64
8   user_id                             142390 non-null  object
9   name                                142382 non-null  object
10  username                             142390 non-null  object
11  followers_count                      142390 non-null  int64
12  user_created_at                     142390 non-null  datetime64[ns]
13  is_verified                          142390 non-null  int64
14  collection_date                     142390 non-null  datetime64[ns]
15  user_pseudo                          142390 non-null  object
16  tweet_id_pseudo                     142390 non-null  object
17  tweet_pseudo                         142390 non-null  object
18  reply_to_user_pseudo                 28575 non-null   object
19  reply_to_tweet_pseudo                28370 non-null   object
dtypes: Int64(1), datetime64[ns](3), int64(3), object(13)
memory usage: 21.9+ MB
```

```
[48]: # Calculando o tamanho do tweet bruto não pseudonimizado
df['raw_tweet_length'] = df['raw_tweet'].str.len()
```

```
[49]: # ===== Remoção das colunas originais e renomeação =====
# Lista de colunas originais para remover
colunas_para_remover = [
    "user_id", "username", "name",
    "tweet_id", "raw_tweet",
    "reply_to_user_id", "reply_to_tweet_id"
]
```

```

# Remove colunas originais (ignora caso não existam)
df = df.drop(columns=colunas_para_remover, errors="ignore")

# Mapeamento de renomeação das colunas pseudonimizadas
mapeamento_renameacao = {

    "user_pseudo": "user_id",
    "tweet_id_pseudo": "tweet_id",
    "tweet_pseudo": "raw_tweet",
    "reply_to_user_pseudo": "reply_to_user_id",
    "reply_to_tweet_pseudo": "reply_to_tweet_id"
}

# Renomeia colunas pseudonimizadas para os nomes originais
df = df.rename(columns=mapeamento_renameacao)

# Definir a ordem das colunas
desired_order = [
    'tweet_id', 'raw_tweet', 'raw_tweet_length', 'link', 'tweet_created_at',
    'reply_to_tweet_id', 'reply_to_user_id', 'retweets', 'likes',
    'user_id', 'followers_count',
    'user_created_at', 'is_verified', 'collection_date'
]

# Reordenar o DataFrame
df = df[desired_order]

```

```
[50]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 142390 entries, 0 to 142389
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   tweet_id              142390 non-null object
1   raw_tweet             142390 non-null object
2   raw_tweet_length      142390 non-null int64
3   link                  17049 non-null  object
4   tweet_created_at      142390 non-null datetime64[ns]
5   reply_to_tweet_id     28370 non-null  object
6   reply_to_user_id      28575 non-null  object
7   retweets              142390 non-null Int64
8   likes                 142390 non-null int64
9   user_id               142390 non-null object
10  followers_count       142390 non-null int64
11  user_created_at       142390 non-null datetime64[ns]
12  is_verified           142390 non-null int64

```

```
13  collection_date    142390 non-null  datetime64[ns]  
dtypes: Int64(1), datetime64[ns](3), int64(4), object(6)  
memory usage: 15.3+ MB
```

B.3 Processamento de Texto dos Tweets

B.3.1 Engenharia de Features Baseadas em Texto (Tweet)

```
[51]: #Extraíndo features dentro do texto do tweet
def extract_elements(text):

    # Extrai URLs que começam com http/https
    raw_urls = re.findall(r'http[s]?://\S+', text)

    # Filtra: sem '...' ou '...', com domínio válido, e com pelo menos um ponto
    urls = []
    for url in raw_urls:
        # Remove '...' e '...' do final da URL
        clean = url.rstrip('...').rstrip('.')
        if len(clean) >= 23 and re.match(r'^https?://[^\s]+\.[^\s]+$', clean):
            urls.append(clean)

    # Extrai hashtags válidas (não permite @ ou letras antes do #)
    raw_hashtags = reg.findall(r'(?<![@\\w])#[\\w\\p{Emoji}]+', text)

    # Remove pontuação final
    clean_hashtags = [tag.rstrip('!.?,...') for tag in raw_hashtags]

    # Filtra hashtags com pelo menos 1 caractere após #
    hashtags = [tag for tag in clean_hashtags if len(tag) > 1]

    # Extrai emojis
    # Regex para capturar sequências de emojis numéricos com keycap(1,2,etc.)
    pattern = re.compile(r'(?:[0-9] )+')
    matches = pattern.findall(text)

    # Remove as sequências para não duplicar quando extrair os demais emojis
    cleaned = pattern.sub('', text)

    # Pega os demais emojis normalmente
    other_emojis = [e['emoji'] for e in emoji.emoji_list(cleaned)]

    # emojis = [c for c in text if c in emoji.EMOJI_DATA]
    emojis = matches + other_emojis

    # Extrai menções
    mentions = re.findall(r'(?<![\\w)@\\w{1,25}(?!\\w)', text)

    # Classifica se é um retweet ou não
    is_retweet = 1 if text.startswith('RT @') else 0

    # Extrai autor original, caso seja um retweet
```

```

original_author = re.findall(r'^RT (@[\s:]+):?', text)

# Remove original_user de mentions se for um retweet
if is_retweet and original_author:
    original_username = original_author[0]
    mentions = [user for user in mentions if user != original_username]

# Verificar se a tweet possuem um tom de alarmismo
words = text.split()
valid_w = [w for w in words if w not in ["RT"]]
upper_ratio = sum(1 for w in valid_w if w.isupper()) / max(1, len(valid_w))
# Caso mais de 50% do texto for composto por letras maiúsculas
# Retorna 1 (verdadeiro), e 0 (falso) caso contrário
is_upper = int(upper_ratio > 0.5)

return {
    'mentions': mentions,
    'urls': urls,
    'emojis': emojis,
    'hashtags': hashtags,
    'is_retweet': is_retweet,
    'original_author': original_author,
    "is_upper": is_upper
}

# Aplicando extração
df_elements = df['raw_tweet'].apply(lambda x: pd.Series(extract_elements(x)))
df = pd.concat([df, df_elements], axis=1)

# Converter original_user de uma lista para um valor único ou None
df['original_author'] = df['original_author'].apply(
    lambda x: x[0] if len(x) > 0 else None
)

```

```

[52]: # Amostragem de dados extraídos
df[['urls', 'emojis', 'hashtags', 'is_retweet', 'is_upper']]
     .sample(10)

```

```

[52]:

```

	urls	emojis	hashtags	is_retweet	is_upper
1662	[]	[]	[]	0	0
131489	[]	[]	[]	1	0
108745	[]	[]	[]	0	0
128326	[]	[]	[]	1	1
91495	[]	[👁️]	[]	1	0
25910	[]	[]	[]	0	0
5216	[]	[]	[#MaríliaÉ77]	1	0

51258	[]	[]	[]	1	0
32771	[]	[]	[]	1	0
1847	[https://t.co/71LfJUWib0]	[]	[]	0	0

```
[53]: # Validando linhas onde pelo menos uma URL ainda contém '...' ou '...'
df[df['urls'].apply(
    lambda lista: any(('...' in url or '...' in url) for url in lista)
)]['urls']
```

```
[53]: Empty DataFrame
Columns: [urls]
Index: []
```

```
[54]: # Adicionar link capturado na coleta aos demais
df['urls'] = [
    ([link] if isinstance(link, str) and link else []) + urls_list
    for link, urls_list in zip(df['link'], df['urls'])
]
df[['link', 'urls']].sample(10)
```

```
[54]:      link urls
133306  NaN  []
28091   NaN  []
690     NaN  []
52900   NaN  []
42512   NaN  []
133639  NaN  []
92564   NaN  []
17653   NaN  []
122026  NaN  []
74515   NaN  []
```

```
[55]: # Validando junção do link capturado na coleta aos demais
df [[
    (link not in urls) if pd.notna(link) else False
    for link, urls in zip(df['link'], df['urls'])
]][['link', 'urls']]
```

```
[55]: Empty DataFrame
Columns: [link, urls]
Index: []
```

```
[56]: # Removendo coluna link
df.drop('link', axis=1, inplace=True)
```

Calculando métricas relacionadas às features extraídas:

```
[57]: # Quantidade de URLs
df['url_count'] = df['urls'].apply(len)

[58]: # Quantidade de hashtags
df['hashtags_count'] = df['hashtags'].apply(lambda x: len(eval(x)) if
↳ isinstance(x, str) and x != '[]' else (len(x) if isinstance(x, list) else 0))

[59]: # Quantidade de menções
df['mentions_count'] = df['mentions'].apply(lambda x: len(eval(x)) if
↳ isinstance(x, str) and x != '[]' else (len(x) if isinstance(x, list) else 0))

[60]: # Quantidade de emojis
df['emoji_count'] = df['emojis'].apply(len)
```

B.3.2 Validação de Conteúdo dos Tweets

B.3.2.1 Identificando Tweets Possivelmente Truncados

```
[61]: # Classificando Tweet com reticências no final
def is_truncated(tweet):
    return (tweet.endswith("...") or tweet.endswith("..."))

df['is_truncated'] = df['raw_tweet'].apply(is_truncated).astype(int)

[62]: # Conta o percentual de cada valor único na coluna 'truncated_tweet'
df['is_truncated'].value_counts(normalize=True)

[62]: is_truncated
1      0.525198
0      0.474802
Name: proportion, dtype: float64
```

Observa-se que aproximadamente 52,5% dos tweets são marcados como possivelmente truncados, enquanto 47,4% não são. Esse percentual considerável sugere que é necessário realizar uma análise cuidadosa antes de tomar qualquer ação, a fim de entender melhor os fatores que influenciam o truncamento dos tweets.

```
[63]: # Amostragem após classificação
df[df['is_truncated'] == 1][['raw_tweet', 'urls', 'is_truncated']].sample(10)

[63]:
```

	raw_tweet	urls	is_truncated
50780	RT @EXT_USER_4651A21C9E61: Só você pode evitar...	[]	1
95979	RT @USER_A83E2154B7EA: A urna possui diversos ...	[]	1
116318	RT @EXT_USER_3C643EF7EFB4: Atenção, amigo: Pat...	[]	1
49749	RT @EXT_USER_45148B1B6F3D: Hoje estava falando...	[]	1
32395	RT @EXT_USER_6B7F10390E04: Nesta eleição presi...	[]	1
81946	RT @EXT_USER_AOB7F0991A51: Você não gosta do P...	[]	1

39847	RT @USER_4D9E47C1EC92: Ontem, às 17h, fui à mi...	[]	1
90823	RT @EXT_USER_A7CA448ADA1C: Hoje deposei meu ...	[]	1
101072	RT @USER_14E8A8278FA5: Achei outra urna, e des...	[]	1
100822	RT @USER_4F9945A69B07: Assim, se constatado qu...	[]	1

```
[64]: # Criar categorias de comprimento para tweets truncados
# Lembrando que 140 caracteres é o tamanho padrao de um tweet
(
    df[df['is_truncated'] == 1]['raw_tweet_length']
    .pipe(
        pd.cut,
        bins=[0, 140, float('inf')],
        labels=['140 caracteres', '>140 caracteres']
    )
    .value_counts()
    .to_frame(name='Quantidade') # Converter para DataFrame
    .assign(
        Percentual=lambda x: (x['Quantidade'] / x['Quantidade'].sum() * 100)
        .round(2)
        .astype(str) + '%'
    )
)
```

```
[64]:
```

	Quantidade	Percentual
raw_tweet_length		
140 caracteres	74113	99.1%
>140 caracteres	670	0.9%

Como a maior parte dos tweets potencialmente truncados possui 140 caracteres, priorizaremos a análise detalhada dessa categoria para identificar possíveis inconsistências ou padrões específicos.

```
[65]: # Analisando uma pequena amostragem dos tweets 140 caracteres
df[(df['is_truncated'] == 1) & (df['raw_tweet_length'] <= 140)]
↳ [['raw_tweet', 'urls', 'is_truncated']].sample(10)
```

```
[65]:
```

	raw_tweet	urls	is_truncated
102531	RT @USER_F1100F3772EE: @USER_EDB7FBODA3E6 Refu...	[]	1
123840	RT @EXT_USER_40DDE6A24B97: O log da urna apont...	[]	1
9864	RT @USER_9135967DEF3A: Olha a Band reconhecend...	[]	1
131618	RT @EXT_USER_8FC8E6D58A1F: A decisão do TSE nã...	[]	1
18423	RT @USER_3BBF5BA931F2: Eu iria de Padre Kelmon...	[]	1
54970	RT @USER_6840D5492C9F: Segundo @EXT_USER_OC63C...	[]	1
118657	RT @EXT_USER_9125032A0F25: AGORA: Em relatório...	[]	1
109576	RT @USER_4535AE1FCBFA: #OsPingosNosIs Por Fe...	[]	1
10206	RT @USER_1BCC7BA1F7A9: atenção trabalhadores/a...	[]	1
5944	RT @USER_34B3DFB9096C: Dia 2 de outubro a orde...	[]	1

NOTA: A célula acima foi executada duas vezes e, em ambas as ocasiões, o número

de RTs superou o de tweets originais.

```
[66]: # Análise do percentual de tweets possivelmente truncados com 140 caracteres,
# comparando retuítes (RTs) e tweets originais
df[(df['is_truncated'] == 1) & (df['raw_tweet_length'] <= 140)
   ].groupby('is_retweet').size()
```

```
[66]: is_retweet
0      544
1    73569
dtype: int64
```

A predominância de retuítes (RTs) entre os casos inconsistentes sugere que a API realizou truncamento automático de RTs com até 140 caracteres, sendo altamente provável que os RTs com mais de 140 caracteres também tenham sido afetados por esse padrão de truncamento.

```
[67]: # Verificando amostragem aleatória de retweets com 140 caracteres
df[(df['is_truncated'] == 1) & (df['raw_tweet_length'] <= 140) &
   (df['is_retweet'] == 1)][['raw_tweet', 'urls', 'is_truncated']].sample(10)
```

```
[67]:
```

	raw_tweet	urls	is_truncated
6884	RT @EXT_USER_31E910E37549: @USER_F93EBFB0E1F6 ...	[]	1
9609	RT @EXT_USER_AC945C887BC0: Repito:\n\nO ÚNICO ...	[]	1
97213	RT @USER_5F1C14C02356: @EXT_USER_D850F3122DF8 ...	[]	1
125442	RT @USER_2395F79F21B5: O estudo do PL mostra q...	[]	1
122091	RT @USER_4535AE1FCBFA: #LinhaDeFrente Tiago ...	[]	1
67303	RT @USER_7D99B738F202: Se a urna deixar e o ma...	[]	1
116284	RT @EXT_USER_3CA2DF92B752: A marinha tava tão ...	[]	1
122921	RT @USER_4535AE1FCBFA: #LinhaDeFrente Tiago ...	[]	1
130218	RT @EXT_USER_8842B7CBAEEB: Sabe o log que o PL...	[]	1
69692	RT @EXT_USER_B615D52D75C2: - Atualizando: Lula...	[]	1

NOTA: A célula acima foi executada duas vezes. Em ambas as ocasiões, identificaram-se diversos casos de palavras incompletas, o que indica que as reticências foram utilizadas não como interrupção intencional do texto, mas sim devido a limitação técnica de caracteres da API.

```
[68]: # Verificando amostragem aleatória de tweets com 140 caracteres
df[(df['is_truncated'] == 1) & (df['raw_tweet_length'] <= 140) &
   (df['is_retweet'] == 0)][['raw_tweet', 'urls', 'is_truncated']].sample(10)
```

```
[68]:
```

	raw_tweet	urls	is_truncated
41250	@EXT_USER_5AAC1DB943B7 Há não precisa mais ir ...	[]	1
19595	@EXT_USER_86D1C426F303 Bolsonaroistas que se pr...	[]	1
39781	@USER_4535AE1FCBFA Não adianta nada buscar apo...	[]	1
70066	@EXT_USER_A01D8D674866 E o Ladraum não manipul...	[]	1
104195	@EXT_USER_7977A82F0AE9 Prender os bandidos...o...	[]	1
115570	@USER_3122333E8A69 @USER_5A2C73791FC3 Da uma o...	[]	1

69559	@EXT_USER_04E1BE090BF1	Maravilha depois de con...	[]	1
15688		Alguém já instalou doom na urna eletrônica? Fi...	[]	1
64642	@EXT_USER_241FDBFEEED	@USER_3CCD23AB19AE Sim,...	[]	1
110406	@EXT_USER_BE7CF8B541A6	Os cara não sabem posta...	[]	1

NOTA: A célula acima foi executada duas vezes. Em ambas as ocasiões, diferentemente dos retweets analisados anteriormente, o uso de reticências não comprometeu o sentido das frases e não houve casos de palavras incompletas, o que sugere uma aplicação deliberada pelos usuários.

A seguir, verificaremos os tweets > 140 caracteres (Tweets e RTs).

```
[69]: # Análise do percentual de tweets possivelmente truncados,
# comparando retuítes (RTs) e tweets originais
df[(df['is_truncated'] == 1) &
    (df['raw_tweet_length'] > 140)].groupby('is_retweet').size()
```

```
[69]: is_retweet
0      658
1       12
dtype: int64
```

```
[70]: # Verificando amostragem aleatória de tweets com > 140 caracteres
df[(df['is_truncated'] == 1) & (df['raw_tweet_length'] > 140) &
    (df['is_retweet'] == 1)][['raw_tweet', 'urls', 'is_truncated']].sample(10)
```

```
[70]:
raw_tweet  urls  is_truncated
73310  RT @USER_64EF00BA7C51: &#8220;600 no Bolso...  []          1
61674  RT @USER_77C3C30D37B1: @USER_A87538EEA374 @EXT...  []          1
44646  RT @USER_808E7410504C: Não captada foi a ABSTE...  []          1
84882  RT @USER_2123330FEFC2: dizem que se digitar 13...  []          1
101888 RT @EXT_USER_878A9C42E201: @USER_001DDDC78280 ...  []          1
24726  RT @USER_CA782FE4E3D8: @EXT_USER_7EFF2409925C ...  []          1
32151  RT @EXT_USER_15420849B3FD: EXCLUSIVO!\n\n> ...  []          1
29274  RT @EXT_USER_D9F4EF0656DD: ATENÇÃO O tema URNA...  []          1
120049 RT @USER_D64A94B8430A: @USER_E10A352A540E Isso...  []          1
5588   RT @USER_40D5AFDEBE16: >@USER_B32354421A2D ...  []          1
```

NOTA: A célula acima foi executada uma única vez, devido ao tamanho reduzido da amostra. Após verificação manual no Twitter/X, constatou-se que o tweet original não apresentava reticências ao final do texto. Portanto, conclui-se que o truncamento identificado é decorrente de limitações técnicas da API.

```
[71]: # Verificando amostragem aleatória de tweets com > 140 caracteres
df[(df['is_truncated'] == 1) & (df['raw_tweet_length'] > 140) &
    (df['is_retweet'] == 0)][['raw_tweet', 'urls', 'is_truncated']].sample(10)
```

```
[71]:
raw_tweet  urls  is_truncated
87392  Fora das linhas democráticas, o campo é infért...  []          1
```

27990	@EXT_USER_17145B41A282	Pois é! Eu confesso que...	[]	1
113220		Agora nem mais é uma questão de urna ou qualqu...	[]	1
122297	@EXT_USER_50ECF3D8C790	@USER_93886F9347F8 Tu l...	[]	1
6266	@EXT_USER_64C39824FE84	Quem dera fosse essa a ...	[]	1
61844		8- Você concorda com os benefícios e monopólio...	[]	1
99106	@EXT_USER_F2FC5FAAF04D	@EXT_USER_A6C9D987187F ...	[]	1
32507	@EXT_USER_743D260F73C9	@EXT_USER_B77D2D818488 ...	[]	1
113742	@EXT_USER_08F129216D41	@EXT_USER_7EA1D3289C47 ...	[]	1
56322	@USER_33A0D753F763	Mas Lucas, uma pergunta lei...	[]	1

NOTA: A célula acima foi executada duas vezes. Em ambas as ocasiões, após verificação manual no Twitter/X, observou-se que os tweets classificados como truncados apresentavam reticências adicionadas deliberadamente pelos usuários, indicando interrupção intencional do texto (não limitação técnica de caracteres). Portanto, os tweets truncados serão reclassificados como não truncados.

```
[72]: # Reclassificando tweets
mask = (df['is_truncated'] == 1) & (df['is_retweet'] == 0)
df.loc[mask, 'is_truncated'] = 0
```

```
[73]: # Verificar tweets não-retweetados com truncated_tweet = 0 e comprimento 140
len(df[(df['is_truncated'] == 1) & (df['raw_tweet_length'] <= 140)
      & (df['is_retweet'] == 0)])
```

[73]: 0

```
[74]: # Verificar tweets não-retweetados com truncated_tweet = 0 e comprimento > 140
len(df[(df['is_truncated'] == 1) & (df['raw_tweet_length'] > 140)
      & (df['is_retweet'] == 0)])
```

[74]: 0

CONCLUSÃO: Muitos tweets longos terminam com reticências, o que poderia indicar truncamento automático. No entanto, na maioria dos casos, as reticências foram usadas intencionalmente pelos autores, seja para sugerir continuidade ou introduzir uma thread. Por isso, esses tweets foram mantidos. Já os retweets foram majoritariamente truncados por limitações da API e serão tratados com cuidado para evitar viés na análise.

B.3.2.2 Tratando Retweets Truncados

```
[75]: # Quantitativo de retweets x tweets
df.groupby('is_retweet').size()
```

```
[75]: is_retweet
0      41659
1     100731
dtype: int64
```

```
[76]: # Quantitativo de retweets truncados x não truncados
df[df['is_retweet'] == 1].groupby('is_truncated').size()
```

```
[76]: is_truncated
0      27150
1      73581
dtype: int64
```

```
[77]: # Em percentual
df[df['is_retweet'] == 1].groupby('is_truncated').size()/len(df)
```

```
[77]: is_truncated
0      0.190674
1      0.516757
dtype: float64
```

Buscar tweet original dentro do dataset:

```
[78]: # Subconjunto de tweets originais (não retweets)
original_tweets_df = df[df['is_retweet'] == 0].copy()

# Função que busca tweet original dentro do dataset,
# baseado na similaridade textual
def find_full_text(row):
    if pd.isna(row['original_author']):
        return np.nan

    # Busca apenas no subconjunto passado (original_tweets_df)
    # Prioriza tweets anteriores
    author_tweets = original_tweets_df[
        (original_tweets_df['user_id'] == row['original_author']) &
        (original_tweets_df['tweet_created_at'] <= row['tweet_created_at'])
    ]['raw_tweet']

    # Remove o prefixo "RT @usuario: " do texto truncado
    truncated_text = re.sub(r'^RT @\w+: ', '', row['raw_tweet']).strip()

    # Compara com os tweets originais do autor
    for original_tweet in author_tweets:
        if SequenceMatcher(None, truncated_text, original_tweet).ratio() > 0.8:
            return original_tweet

    return np.nan

# Máscara para tweets truncados que são retweets
mask = (df['is_retweet'] == 1) & (df['is_truncated'] == 1)

# Inicializa a nova coluna
```

```
df['full_text'] = np.nan

# Aplica a função somente nos truncados
df.loc[mask, 'full_text'] = df.loc[mask].swifter.apply(find_full_text, axis=1)
```

Pandas Apply: 0% | 0/73581 [00:00<?, ?it/s]

```
[79]: # Total de valores nulos na coluna 'full_text'
nulos = df[(df['is_retweet'] == 1) & (df['is_truncated'] == 1)
           & (df['full_text'].isna())].shape[0]

# Total de valores não nulos na coluna 'full_text'
nao_nulos = df[(df['is_retweet'] == 1) & (df['is_truncated'] == 1)
               & (df['full_text'].notna())].shape[0]

print(f"Valores nulos: {nulos}")
print(f"Valores não nulos: {nao_nulos}")
```

Valores nulos: 71737

Valores não nulos: 1844

```
[80]: # Amostragem de tweets encontrados
df[(df['full_text'].notna()) & (df['is_retweet'] == 1) &
   (df['is_truncated'] == 1)][['raw_tweet', 'full_text']].sample(10)
```

```
[80]:                                     raw_tweet \

8113      RT @USER_7CB031F59F7A: Em 12 dias os brasileir...
72793      RT @USER_3CA88B48D40D: BOLSONARO VAI ROUBAR SU...
64992      RT @USER_A4A346917472: Inacreditável q 51 milh...
8075       RT @USER_1F7381E3E343: Ouvimos falar por aqui ...
59480      RT @USER_645A5BB0774D: Brasil com melhor proje...
32745      RT @USER_OB1A2771E256: Lula retorna ao Ceará c...
18845      RT @USER_OA45F12EB74C: Eles não existem sem o ...
76589      RT @USER_AE56A9173AD2: E pensar que no próximo...
115259     RT @USER_B271BF5CC404: Documento do Ministério...
33164      RT @USER_OB1A2771E256: Lula retorna ao Ceará c...

                                     full_text

8113      Em 12 dias os brasileiros ou a urna eletrônica...
72793      BOLSONARO VAI ROUBAR SUA APOSENTADORIA \nBOLS...
64992      Inacreditável q 51 milhões de brasileiros tenh...
8075      Ouvimos falar por aqui que faltam 13 dias pra ...
59480      Brasil com melhor projeção de PIB e menor infl...
32745      Lula retorna ao Ceará com Elmano e Camilo em c...
18845      Eles não existem sem o Lula.\nE o Lula hein???...
76589      E pensar que no próximo domingo estaremos crav...
115259     Documento do Ministério da Defesa afirma que o...
33164      Lula retorna ao Ceará com Elmano e Camilo em c...
```

```
[81]: # Só para as linhas onde encontramos o full_text
mask = df['full_text'].notna()

# Extrai o usuário do texto original
user = df.loc[mask, 'raw_tweet'].str.extract(r'^RT @(\w+):')

# Reconstrói o tweet completo: RT @usuario: full_text
df.loc[mask, 'raw_tweet'] = 'RT @' + user[0] + ': ' + df.loc[mask, 'full_text']
```

```
[82]: # Reclassificando tweets
df['is_truncated'] = df['raw_tweet'].apply(is_truncated).astype(int)
```

```
[83]: # Removendo coluna 'full_text'
df.drop(columns='full_text', inplace=True)
```

```
[84]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 142390 entries, 0 to 142389
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype
---  -
0   tweet_id              142390 non-null  object
1   raw_tweet             142390 non-null  object
2   raw_tweet_length      142390 non-null  int64
3   tweet_created_at      142390 non-null  datetime64[ns]
4   reply_to_tweet_id     28370 non-null   object
5   reply_to_user_id      28575 non-null   object
6   retweets              142390 non-null  Int64
7   likes                142390 non-null  int64
8   user_id              142390 non-null  object
9   followers_count       142390 non-null  int64
10  user_created_at       142390 non-null  datetime64[ns]
11  is_verified           142390 non-null  int64
12  collection_date       142390 non-null  datetime64[ns]
13  mentions              142390 non-null  object
14  urls                  142390 non-null  object
15  emojis                142390 non-null  object
16  hashtags              142390 non-null  object
17  is_retweet            142390 non-null  int64
18  original_author       100731 non-null   object
19  is_upper              142390 non-null  int64
20  url_count             142390 non-null  int64
21  hashtags_count        142390 non-null  int64
22  mentions_count        142390 non-null  int64
23  emoji_count           142390 non-null  int64
24  is_truncated          142390 non-null  int64
dtypes: Int64(1), datetime64[ns](3), int64(11), object(10)
```

```
memory usage: 27.3+ MB
```

B.3.2.3 Identificando Ausência de Palavras-chave (Outliers)

NOTA: Os critérios de exclusão serão baseados no potencial informativo do tweet ou retweet. Assim, publicações sem palavras-chave, mas que contenham links, serão mantidas.

```
[85]: # Classificando Tweet que possui ou não as palavras-chave usadas como filtro na coleta
def contem_urna(texto):
    return int(bool(re.search(r'(?i)(#|\b)urna(s?)(\s*eletronicas?)?\b', texto)))

df['has keywords'] = df['raw tweet'].apply(contem_urna)
```

```
[86]: # Conta o percentual de cada valor único na coluna 'has_keywords'.
df['has_keywords'].value_counts(normalize=True)
```

```
[86]: has_keywords
      1    0.774977
      0    0.225023
      Name: proportion, dtype: float64
```

Cerca de 77,49% dos tweets possuem palavras-chave, enquanto 22,50% não as contêm. Dado o percentual relativamente alto de tweets sem palavras-chave, a exclusão desses registros sem uma análise prévia não é recomendada, pois isso poderia resultar na perda de uma quantidade significativa de dados relevantes.

```
[87]: df[df['has_keywords']== 0][['raw_tweet','urls','has_keywords']]
```

```
[87]:
```

		raw_tweet \
36	@EXT_USER_5719073B2C1D @EXT_USER_1CAB813EE2E4	...
37	@EXT_USER_5719073B2C1D @EXT_USER_1CAB813EE2E4	
121	@USER_C45CB755EEBB @EXT_USER_2E9A690681B7 @EXT...	
126	@EXT_USER_7AC2AE268640 @EXT_USER_3080E627A984	
201	@EXT_USER_D6851BD6A423 @EXT_USER_E6B728A535AD	...
...		...
142100	RT @EXT_USER_8FC8E6D58A1F: (1/3) Vou tentar re...	
142192	RT @EXT_USER_073468D94D51: Nas eleições, prova...	
142312	RT @USER_4E7B25B2955F: @USER_A87538EEA374 Eu f...	
142374	RT @USER_7751DB7A3A12: Perguntas para quem já ...	
142386	RT @USER_7751DB7A3A12: Perguntas para quem já ...	
	urls	has_keywords
36	[https://t.co/ovy8L7nqLs]	0
37	[https://t.co/cr5KRyh4gw]	0
121	[https://t.co/K6djKJeRGR]	0

126	[https://t.co/VSQw12vnNe]	0
201	[https://t.co/6Wuu7X8x7e]	0
...	...	...
142100	[]	0
142192	[]	0
142312	[]	0
142374	[]	0
142386	[]	0

[32041 rows x 3 columns]

```
[88]: # Filtrando tweets muito curtos e com urls
df[(df['has_keywords']== 0) & (df['urls'].apply(lambda x: len(x) >= 1)) &
    (df['raw_tweet_length'] < 50)][['raw_tweet', 'urls', 'has_keywords']]
```

```
[88]:                                     raw_tweet \
36      @EXT_USER_5719073B2C1D @EXT_USER_1CAB813EE2E4 ...
37      @EXT_USER_5719073B2C1D @EXT_USER_1CAB813EE2E4
126      @EXT_USER_7AC2AE268640 @EXT_USER_3080E627A984
356      @EXT_USER_8335FA30F33A @EXT_USER_97DC9A95E441 ...
421      @EXT_USER_077641B3C03F @EXT_USER_0FB401915AEA ...
...
135281      Recomendando ler essa reflexão.\n\n
135910      Interessante. Sugiro a leitura
137429 @EXT_USER_373B02C466CB @USER_6DEC4BBAA7CD Faz ...
137824 @EXT_USER_619325A653CC @EXT_USER_C4768F069EC3 ...
137959      @EXT_USER_79EB25ABFC6F Não, PERDEU !!!\n\n

                                     urls  has_keywords
36      [https://t.co/ovy8L7nqLs]          0
37      [https://t.co/cr5KRyh4gw]          0
126      [https://t.co/VSQw12vnNe]          0
356      [https://t.co/h0Hn4VJml6]          0
421      [https://t.co/r1nYERUPVm]          0
...
135281 [https://t.co/6g0Tbx28aT]          0
135910 [https://t.co/NtYJrfnCUG]          0
137429 [https://t.co/RdgbdzrFTN]          0
137824 [https://t.co/Ecom4NWna8]          0
137959 [https://t.co/un3Co3MMG6]          0
```

[95 rows x 3 columns]

Observou-se que, em tweets curtos que não apresentam palavras-chave no corpo do texto, essas palavras muitas vezes estão presentes nas URLs dos links encurtados, após o redirecionamento. Embora a análise tenha focado inicialmente nesses tweets mais breves, é razoável supor que o mesmo comportamento ocorra também em tweets mais longos que contenham links. Por essa razão, optou-se por manter esses registros para

análises futuras, considerando a relevância potencial das palavras-chave presentes nas URLs.

```
[89]: # Filtrando tweets muito curtos e sem links
df[(df['has_keywords']== 0) & (df['urls'].apply(lambda x: len(x) < 1)) &
   (df['raw_tweet_length'] < 50) ][['raw_tweet','urls','has_keywords']]
```

```
[89]:
```

	raw_tweet	urls	has_keywords
7818	@USER_136D29AEEF9A Acha	[]	0
17045	@USER_CA0C71F234E7   achaaa	[]	0
18960	@USER_136D29AEEF9A US Fara  	[]	0
47393	@USER_136D29AEEF9A acha g 	[]	0
51835	@USER_136D29AEEF9A Faraaaaaaa      	[]	0
53891	@USER_136D29AEEF9A Eloowww	[]	0
61129	@EXT_USER_4278BFBF1513 hlww 	[]	0
74667	@USER_136D29AEEF9A Acha ni?	[]	0
86871	@USER_136D29AEEF9A Naii aisa naii ho ga	[]	0
123177	@USER_136D29AEEF9A Acha 	[]	0

```
[90]: # Removendo tweets muito curtos e sem links
df = df[~((df['has_keywords'] == 0) & (df['urls'].apply(lambda x: len(x) < 1)) &
   (df['raw_tweet_length'] < 50))]
df = df.reset_index(drop=True)
```

```
[91]: # Validando exclusão
df[(df['has_keywords']== 0) & (df['urls'].apply(lambda x: len(x) < 1)) &
   (df['raw_tweet_length'] < 50) ][
   ['raw_tweet','urls','has_keywords','is_truncated']]
```

```
[91]: Empty DataFrame
Columns: [raw_tweet, urls, has_keywords, is_truncated]
Index: []
```

```
[92]: # Filtrando tweets medianos, sem links e sem palavras-chave
df[(df['has_keywords']== 0) & (df['urls'].apply(lambda x: len(x) < 1)) &
   (df['raw_tweet_length'] <= 100) ][
   ['raw_tweet','urls','has_keywords']]
```

```
[92]:
```

	raw_tweet	urls	has_keywords
77397	@EXT_USER_D5E9F2442B56 No primeiro turno a úrn...	[]	0
83223	@USER_290A93A72872's account has been withheld...	[]	0
98839	@EXT_USER_562943E1BAEE's account has been with...	[]	0
98842	@EXT_USER_562943E1BAEE's account has been with...	[]	0
98865	@EXT_USER_562943E1BAEE's account has been with...	[]	0
...	...	...	...
134288	@EXT_USER_7498B96FCB21's account has been with...	[]	0
134911	@EXT_USER_DD3E4C837E14's account has been with...	[]	0
135922	@EXT_USER_7498B96FCB21's account has been with...	[]	0

137159	@USER_9A0074795F58's account has been withheld...	[]	0
137909	@USER_2395F79F21B5's account has been withheld...	[]	0

[99 rows x 3 columns]

NOTA: As contas dos usuários autores dos tweets acima foram retidas a pedido da Justiça brasileira.

```
[93]: # Classificando tweets com conta retida.
df['is_withheld'] = df['raw_tweet'].str.contains(
    "has been withheld in Brazil in response to a legal demand",
    na=False).astype(int)
```

```
[94]: # Filtrando tweets 140 caracteres
df_f = df[
    (df['has_keywords'] == 0) & # Tweets sem palavras-chave
    (df['urls'].apply(lambda x: len(x) < 1)) & # Sem links externos
    (df['is_withheld'] == 0) & # Sem tweets bloqueados/retidos
    (df['raw_tweet_length'] <= 140) & # Texto com mais de 140 caracteres
    (df['is_retweet'] == 0)
][['raw_tweet', 'urls', 'has_keywords']]

df_f
```

```
[94]:                                     raw_tweet urls  has_keywords
77397  @EXT_USER_D5E9F2442B56 No primeiro turno a urn... []          0
```

```
[95]: # Removendo tweets
df = df[~df.index.isin(df_f.index)]
df = df.reset_index(drop=True)
```

```
[96]: # Filtrando tweets 140 caracteres
df_f = df[
    (df['has_keywords'] == 0) & # Tweets sem palavras-chave
    (df['urls'].apply(lambda x: len(x) >= 1)) & # Com links externos
    (df['is_withheld'] == 0) & # Sem tweets bloqueados/retidos
    (df['raw_tweet_length'] <= 140) & # Texto com mais de 140 caracteres
    (df['is_retweet'] == 0)
][['raw_tweet', 'urls', 'has_keywords']]

df_f
```

```
[96]:                                     raw_tweet \
36      @EXT_USER_5719073B2C1D @EXT_USER_1CAB813EE2E4 ...
37      @EXT_USER_5719073B2C1D @EXT_USER_1CAB813EE2E4
126      @EXT_USER_7AC2AE268640 @EXT_USER_3080E627A984
201      @EXT_USER_D6851BD6A423 @EXT_USER_E6B728A535AD ...
331      Para saber mais sobre as etapas que ocorrem du...
```

```

...
140190 @EXT_USER_F40999C9C592 Pen drive. Que coisa ul...
140460 @EXT_USER_73DFD9D4C616 @EXT_USER_9626C0CBFB61 ...
140583 @USER_D29D03FE2A93 @USER_7D99B738F202 Cara, o ...
140714 Hackers invadem sistema da Justiça Eleitoral e...
141011 @EXT_USER_871DE414ABD4 @USER_07473D6D4251 \n\n...

          urls  has_keywords
36          [https://t.co/ovy8L7nqLs]          0
37          [https://t.co/cr5KRyh4gw]          0
126         [https://t.co/VSQw12vnNe]          0
201         [https://t.co/6Wuu7X8x7e]          0
331  [https://t.co/5H9g7bYiCN, https://t.co/Hi7ewn4...          0
...
140190          [https://t.co/CLp1L9DcVa]          0
140460          [https://t.co/JcUG2uacS4]          0
140583          [https://t.co/IzXIYzM2LE]          0
140714          [https://t.co/HqoQONcwkj]          0
141011  [https://t.co/mmSoyHQfAP, https://t.co/AciLRph...          0

```

[299 rows x 3 columns]

NOTA: Como foi dito anteriormente, apesar de não conter as palavras-chaves no corpo do tweets, estas foram identificadas nas URLs.

```

[97]: # Filtrando tweets >140 caracteres
df_f = df[
    (df['has_keywords'] == 0) & # Tweets sem palavras-chave
    (df['urls'].apply(lambda x: len(x) < 1)) & # Sem links externos
    (df['is_withheld'] == 0) & # Sem tweets bloqueados/retidos
    (df['raw_tweet_length'] > 140) & # Texto com mais de 140 caracteres
    (df['is_retweet'] == 0)
][['raw_tweet', 'urls', 'has_keywords']]

df_f

```

```

[97]:          raw_tweet  urls  has_keywords
71945  521 mil pessoas anularam o vøtø para Gøvørnødø...  []          0
90323  @EXT_USER_1072EC37D652 Bolsonaro caiu pq escut...  []          0
96848  @USER_B0249C172B21 @USER_A83E2154B7EA Tenho a ...  []          0

```

```

[98]: # Removendo tweets
df = df[~df.index.isin(df_f.index)]
df = df.reset_index(drop=True)

```

Verificando Retweets sem Palavras-chave:

```
[99]: # Filtrando retweets 140 caracteres
df_f = df[
    (df['has_keywords'] == 0) &
    (df['urls'].apply(lambda x: len(x) < 1)) &
    (df['is_withheld'] == 0) &
    (df['raw_tweet_length'] <= 140) &
    (df['is_retweet'] == 1) &
    (df['is_truncated'] == 0)
][['raw_tweet', 'urls', 'has_keywords', 'is_truncated']]

df_f
```

```
[99]:
```

	raw_tweet	urls	has_keywords	\
10277	RT @USER_B77573D5135E: [Podcast] Cientista po...	[]	0	
11084	RT @EXT_USER_4EC0707C765C: Nas eleições deste ...	[]	0	
17239	RT @USER_236614D67F9E: Sem resposta alguma, pe...	[]	0	
17242	RT @USER_236614D67F9E: Sem resposta alguma, pe...	[]	0	
17329	RT @USER_236614D67F9E: Sem resposta alguma, pe...	[]	0	
...	...	...	...	
107180	RT @EXT_USER_7382ADF70842: O Valor está faland...	[]	0	
107210	RT @EXT_USER_7382ADF70842: O Valor está faland...	[]	0	
139401	RT @USER_7751DB7A3A12: @USER_8641784A80EC Não ...	[]	0	
140313	RT @USER_7751DB7A3A12: @USER_8641784A80EC Não ...	[]	0	
141576	RT @USER_7751DB7A3A12: @USER_8641784A80EC Não ...	[]	0	

	is_truncated
10277	0
11084	0
17239	0
17242	0
17329	0
...	...
107180	0
107210	0
139401	0
140313	0
141576	0

[125 rows x 4 columns]

```
[100]: # Removendo retweets
df = df[~df.index.isin(df_f.index)]
df = df.reset_index(drop=True)
```

```
[101]: # Filtrando retweets >140 caracteres
df_f = df[
    (df['has_keywords'] == 0) &
```

```

(df['urls'].apply(lambda x: len(x) < 1)) &
(df['is_withheld'] == 0) &
(df['raw_tweet_length'] > 140) &
(df['is_retweet'] == 1) &
(df['is_truncated'] == 0)
][['raw_tweet', 'urls', 'has_keywords', 'is_truncated']]

df_f

```

[101]: Empty DataFrame
Columns: [raw_tweet, urls, has_keywords, is_truncated]
Index: []

```

[102]: # Filtrando retweets 140 caracteres
df_f = df[
    (df['has_keywords'] == 0) &
    (df['urls'].apply(lambda x: len(x) < 1)) &
    (df['is_withheld'] == 0) &
    (df['raw_tweet_length'] <= 140) &
    (df['is_retweet'] == 1) &
    (df['is_truncated'] == 1)
][['raw_tweet', 'urls', 'has_keywords']]

df_f

```

```

[102]:

```

	raw_tweet	urls	has_keywords
4473	RT @EXT_USER_B27BD2E39AA4: Quero, sim, ser a p...	[]	0
4493	RT @USER_FDD21C3B0E39: @EXT_USER_B4340B87AF8E ...	[]	0
4498	RT @EXT_USER_06530D2A753A: Bolsonaro sempre ch...	[]	0
4521	RT @USER_2395F79F21B5: A fase de adaptação à r...	[]	0
4530	RT @USER_C06F1FB078AC: Obs: a aprovação da lei...	[]	0
...	...	...	...
141961	RT @EXT_USER_8FC8E6D58A1F: (1/3) Vou tentar re...	[]	0
142053	RT @EXT_USER_073468D94D51: Nas eleições, prova...	[]	0
142173	RT @USER_4E7B25B2955F: @USER_A87538EEA374 Eu f...	[]	0
142235	RT @USER_7751DB7A3A12: Perguntas para quem já ...	[]	0
142247	RT @USER_7751DB7A3A12: Perguntas para quem já ...	[]	0

[30965 rows x 3 columns]

```

[103]: # Removendo retweets
df = df[~df.index.isin(df_f.index)]
df = df.reset_index(drop=True)

```

```

[104]: # Validando
df[df['is_retweet'] == 1].groupby('is_truncated').size()

```

```
[104]: is_truncated
0      28851
1      40790
dtype: int64
```

```
[105]: # Validando
df.groupby('is_retweet').size()
```

```
[105]: is_retweet
0      41645
1      69641
dtype: int64
```

```
[106]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 111286 entries, 0 to 111285
Data columns (total 27 columns):
#   Column                Non-Null Count  Dtype
---  -
0   tweet_id              111286 non-null object
1   raw_tweet             111286 non-null object
2   raw_tweet_length      111286 non-null int64
3   tweet_created_at      111286 non-null datetime64[ns]
4   reply_to_tweet_id     28357 non-null  object
5   reply_to_user_id      28562 non-null  object
6   retweets              111286 non-null Int64
7   likes                 111286 non-null int64
8   user_id               111286 non-null object
9   followers_count       111286 non-null int64
10  user_created_at        111286 non-null datetime64[ns]
11  is_verified            111286 non-null int64
12  collection_date        111286 non-null datetime64[ns]
13  mentions               111286 non-null object
14  urls                   111286 non-null object
15  emojis                 111286 non-null object
16  hashtags               111286 non-null object
17  is_retweet             111286 non-null int64
18  original_author        69641 non-null  object
19  is_upper               111286 non-null int64
20  url_count              111286 non-null int64
21  hashtags_count         111286 non-null int64
22  mentions_count         111286 non-null int64
23  emoji_count            111286 non-null int64
24  is_truncated           111286 non-null int64
25  has_keywords           111286 non-null int64
26  is_withheld            111286 non-null int64
dtypes: Int64(1), datetime64[ns](3), int64(13), object(10)
```

memory usage: 23.0+ MB

B.3.3 Remoção de Outliers Semânticos

O objetivo de verificar se todas as menções à palavra ‘urna’ estavam relacionadas ao objeto de estudo (urna eletrônica).

```
[ ]: palavras_outros_sentidos = [
    "funer[áa]ria", "mortu[áa]ria", "crema[cç][aã]o", "sorteio", "decorativa"
]

pattern_urna_outros = r"\burna\b.*\b(?:{ })\b".format("|".
    ↪join(palavras_outros_sentidos))

mask_outros = df['raw_tweet'].str.contains(pattern_urna_outros, flags=re.
    ↪IGNORECASE, na=False)
mask_eletronica = df['raw_tweet'].str.contains(r"\beletr[oô]nic[ao]?b",
    ↪flags=re.IGNORECASE, na=False)

mask_final = mask_outros & ~mask_eletronica
tweets_urna_outros = df[mask_final]
tweets_urna_outros[['tweet_id', 'raw_tweet']]
```

```
[ ]: tweet_id      raw_tweet
4659    TWEET_6C0E632DDE4D  @EXT_USER_7EE3A068B2B7 Pego os santinhos no ch...
5724    TWEET_EDF212F3BB0D  Bolsonaro está em Londres fazendo campanha de ...
7727    TWEET_07AB73C9656B  @EXT_USER_9E7F1841AAAA Na urna funerária!! É L...
8726    TWEET_799087184C1D  Promoção Casa Campeã Disensa \n\nCompre nas lo...
10134   TWEET_D9EBFE8EA8B4  O cara sai de seu país e inventa de fazer
    ↪prop...      ...      ...
    ↪      ...
103608  TWEET_A8BC8C7857EF  @USER_A87538EEA374 Olhar impressão de boletim ...
104653  TWEET_4A0EAC3FA575  @EXT_USER_6630F41C7A5A @EXT_USER_BE7CF8B541A6 ...
104752  TWEET_63DB9E06BDBF  Na ausência de melhores transportadores e carr...
104758  TWEET_CB6762FA8BC6  Na ausência de melhores transportadores e carr...
108521  TWEET_89A4E6035F1D  @USER_D5970876CB03 Qual será o modelo da urna ...

[70 rows x 2 columns]
```

```
[ ]: # Remover tweets inválidos
df = df[~mask_final]
```

B.3.4 Pré-processamento de Texto

```
[107]: # Pré-processando tweets (Para aplicar técnicas de NLP)
def preprocess_tweet(text):
    text = re.sub(r"http\S+|www.\S+", "", text) # Remove URLs
    text = re.sub(r"@w+", "", text) # Remove @menções
```

```

text = re.sub(r"#", "", text) # Remove "#" mas mantém a palavra
text = re.sub(r'^RT\b[\s:]*([@\[\].*?:\s*)?', "", text) # Remove RTs
text = emoji.replace_emoji(text, replace='') # Remove emojis
text = re.sub(r"\s{2,}", " ", text).strip() # Espaços extras
return text

```

```

[108]: # Aplicando função de pré-processamento
df["preprocessed_tweet"] = df["raw_tweet"].apply(preprocess_tweet)

```

```

[109]: # Amostragem após pré-processamento
df[['preprocessed_tweet']].sample(10)

```

```

[109]:                                     preprocessed_tweet
28385  Boletins de Urna Liberados NOVA ZELANDIA - Lul...
5847   Eu concordo ctg q é um absurdo gigante essa na...
33135  Fui fiscal de urna hoje. O que eu vi? Urnas fr...
18246  foi oq ele disse...\n"vou apertar 22 COM MAIS ...
46397  Tem tb a mentira enviada aos WhatsApp dos idos...
37780  Bom diaaaaaaaá minha Thukinha É Treze na Urna e...
88219  LOOLA FOI NOMEADO PELO TSE\nForam 10.000 urnas...
12884                                     é 22 na urna\ndigitaaaaaaaaaaaaaaaaaaaa
75216  Como pode uma mesma região ou município difere...
80490  Auditoria do TCU não identifica falhas em urna...

```

B.4 Engenharia de Features

B.4.1 Extração de Métricas e Atributos do Tweet

```
[110]: # Taxa de engajamento
      """
      Fórmula baseada em:
      https://www.rdstation.com/blog/marketing/taxa-de-engajamento/,
      https://www.mailmodo.com/tools/twitter-engagement-rate-calculator/

      """
      # Calcular engajamento bruto
      df['engagement_raw'] = df['likes'] + df['retweets']

      # Normalizar por seguidores
      df['engagement_rate'] = (df['engagement_raw'] / df['followers_count']).
                              replace(0, 1)).round(4)

      # Aplicar log para reduzir outliers e normalizar para escala 0-100
      df['engagement_index'] = (
          (np.log1p(df['engagement_rate']) - np.log1p(df['engagement_rate']).min()) /
          (np.log1p(df['engagement_rate']).max() - np.log1p(df['engagement_rate']).min())
      ) * 100

      df['engagement_index'] = df['engagement_index'].round(2)
```

```
[111]: # Amostragem após pré-processamento
      df[["engagement_raw", "engagement_rate", "engagement_index"]].sample(10)
```

```
[111]:
```

	engagement_raw	engagement_rate	engagement_index
34896	692	5.4488	18.26
3602	0	0.0	0.0
51426	82	0.0098	0.1
52424	368	0.1548	1.41
1985	0	0.0	0.0
69829	0	0.0	0.0
77451	5272	112.1702	46.34
106123	0	0.0	0.0
59473	0	0.0	0.0
79051	20	0.2353	2.07

```
[112]: # Idade da conta
      df['account_age_days'] = (df['collection_date'] - df['user_created_at']).dt.days
```

```
[113]: # Tamanho dos tweets
      df['tweet_length'] = df['preprocessed_tweet'].str.len()
```

```
[114]: # Referência : https://www.tre-sp.jus.br/eleicoes/eleicoes-anteriores/eleicoes-2022/calendario-eleitoral-resumido
def categorizar_fase_eleitoral(data):

    data = data.date()

    if data < pd.to_datetime('2022-10-02').date():
        return "Campanha 1º Turno"
    elif pd.to_datetime('2022-10-02').date() == data:
        return "1º Turno (Votação)"
    elif (data > pd.to_datetime('2022-10-02').date()) and (data < pd.to_datetime('2022-10-30').date()):
        return "Campanha 2º Turno"
    elif pd.to_datetime('2022-10-30').date() == data:
        return "2º Turno (Votação)"
    elif data > pd.to_datetime('2022-10-30').date():
        return "Pós-eleição"

# Aplicando função de categorização
df['electoral_phase'] = df['tweet_created_at'].apply(categorizar_fase_eleitoral)
```

```
[115]: # Amostragem após categorização
df[['account_age_days', 'engagement_rate', 'electoral_phase']].sample(10)
```

```
[115]:      account_age_days  engagement_rate  electoral_phase
78303             996          170.9872        Pós-eleição
16138              18           0.0442    Campanha 1º Turno
44585            1366              0.0    Campanha 2º Turno
49194              83           0.1497    Campanha 2º Turno
97469             643          251.5789        Pós-eleição
75389            1506              0.0        Pós-eleição
94460             945           0.0109        Pós-eleição
75076            1767              0.0        Pós-eleição
42931            2556           0.0019    Campanha 2º Turno
31552            4933              0.0    Campanha 2º Turno
```

Validando Filtro da Fase Eleitoral (Opcional):

```
[116]: # Amostragem após categorização
df[df['tweet_created_at'].dt.date < pd.to_datetime('2022-10-02').date()
   ][['tweet_created_at', 'electoral_phase']].sample(10)
```

```
[116]:      tweet_created_at  electoral_phase
11140 2022-09-23 00:39:38    Campanha 1º Turno
18423 2022-09-27 07:04:24    Campanha 1º Turno
1407  2022-09-06 16:46:50    Campanha 1º Turno
22176 2022-09-29 01:56:56    Campanha 1º Turno
24640 2022-09-30 07:56:42    Campanha 1º Turno
```

```

20620 2022-09-28 08:09:05 Campanha 1º Turno
11735 2022-09-23 07:20:42 Campanha 1º Turno
13137 2022-09-24 01:24:54 Campanha 1º Turno
14301 2022-09-25 00:11:58 Campanha 1º Turno
13547 2022-09-24 10:11:45 Campanha 1º Turno

```

```

[117]: # Amostragem após categorização
df[df['tweet_created_at'].dt.date == pd.to_datetime('2022-10-02').date()
    ][['tweet_created_at', 'electoral_phase']].sample(10)

```

```

[117]:      tweet_created_at      electoral_phase
28016 2022-10-02 05:08:43 1º Turno (Votação)
29637 2022-10-02 23:52:50 1º Turno (Votação)
28582 2022-10-02 11:45:33 1º Turno (Votação)
27976 2022-10-02 04:43:09 1º Turno (Votação)
29450 2022-10-02 21:36:36 1º Turno (Votação)
28832 2022-10-02 14:32:12 1º Turno (Votação)
28628 2022-10-02 12:19:20 1º Turno (Votação)
29205 2022-10-02 18:33:50 1º Turno (Votação)
29144 2022-10-02 17:54:06 1º Turno (Votação)
28203 2022-10-02 07:08:59 1º Turno (Votação)

```

```

[118]: # Amostragem após categorização
df[(df['tweet_created_at'].dt.date > pd.to_datetime('2022-10-02').date()) &
    (df['tweet_created_at'].dt.date < pd.to_datetime('2022-10-30').date())
    ][['tweet_created_at', 'electoral_phase']].sample(10)

```

```

[118]:      tweet_created_at      electoral_phase
58983 2022-10-23 12:13:50 Campanha 2º Turno
55810 2022-10-21 02:52:11 Campanha 2º Turno
60549 2022-10-25 00:20:35 Campanha 2º Turno
42960 2022-10-10 18:16:26 Campanha 2º Turno
56760 2022-10-21 22:52:41 Campanha 2º Turno
63495 2022-10-27 09:50:41 Campanha 2º Turno
43243 2022-10-10 23:42:10 Campanha 2º Turno
39901 2022-10-08 13:02:34 Campanha 2º Turno
43972 2022-10-11 13:08:25 Campanha 2º Turno
56626 2022-10-21 20:29:10 Campanha 2º Turno

```

```

[119]: # Amostragem após categorização
df[df['tweet_created_at'].dt.date == pd.to_datetime('2022-10-30').date()
    ][['tweet_created_at', 'electoral_phase']].sample(10)

```

```

[119]:      tweet_created_at      electoral_phase
66616 2022-10-30 06:12:54 2º Turno (Votação)
66300 2022-10-30 01:51:41 2º Turno (Votação)
66169 2022-10-30 00:03:07 2º Turno (Votação)

```

66442	2022-10-30	03:50:46	2º Turno (Votação)
67698	2022-10-30	23:18:12	2º Turno (Votação)
67657	2022-10-30	22:46:06	2º Turno (Votação)
66998	2022-10-30	11:35:10	2º Turno (Votação)
67518	2022-10-30	20:51:26	2º Turno (Votação)
66403	2022-10-30	03:12:57	2º Turno (Votação)
66919	2022-10-30	10:31:06	2º Turno (Votação)

```
[120]: # Amostragem após categorização
df[df['tweet_created_at'].dt.date > pd.to_datetime('2022-10-30').date()
][['tweet_created_at', 'electoral_phase']].sample(10)
```

```
[120]:      tweet_created_at  electoral_phase
72878  2022-11-03 14:21:20      Pós-eleição
106835 2022-11-28 18:53:55      Pós-eleição
94037  2022-11-17 19:48:51      Pós-eleição
95730  2022-11-19 00:28:04      Pós-eleição
108528 2022-11-30 04:51:17      Pós-eleição
67818  2022-10-31 00:56:55      Pós-eleição
70405  2022-11-01 13:46:31      Pós-eleição
96777  2022-11-19 17:19:04      Pós-eleição
80378  2022-11-08 23:27:36      Pós-eleição
68120  2022-10-31 05:11:34      Pós-eleição
```

B.4.2 Expansão de URLs Encurtadas

```
[121]: # Quantidade de linhas com URLs
df['urls'].apply(lambda x: isinstance(x, list) and len(x) > 0).sum()
```

```
[121]: np.int64(17025)
```

O código a seguir foi desenvolvido de forma empírica, com base na lógica utilizada pelo site WhereGoes, seguindo a mesma abordagem de análise e rastreamento de redirecionamentos. Link: <https://wheregoes.com/trace/20253102014/>

 **WHEREGOES**
FROM CLICK TO LOAD

URL Redirect Checker DNS Lookup

Date Traced: 2025-05-01 16:31:48 GMT

User Agent:

#	Code	Requested URL
✓	301	https://t.co/xLmmwla3x9 ▪ Redirects: 2
1	301	  https://t.co/xLmmwla3x9 <u>301 Redirect</u>
2	302	  https://twitter.com/omarxista/status/1592251486700179456/photo/1 <u>302 Redirect</u>
3	400	  https://x.com/omarxista/status/1592251486700179456/photo/1 <u>Error: Status Code 400</u>

NOTICE: The results above may vary depending on the user-agent used during the test and whether or not the responding server is blocking WhereGoes. If you are seeing a 403 response code, the trace might have been blocked.

```
[122]: # Iniciando cache
cache = {}

def expand_url(url):
    # Verifica se já está no cache
    if url in cache:
        return cache[url]
    try:
        response = requests.head(url, allow_redirects=False, timeout=5)
        # Códigos de redirecionamento:
```

```

# https://www.baeldung.com/cs/redirection-status-codes
if response.status_code in (301, 302, 303, 307, 308):
    # Obtém a URL de redirecionamento a partir do cabeçalho 'Location'
    location = response.headers['location']
    """
    Chama recursivamente a função expand_url para seguir a cadeia de
    redirecionamentos. Caso o 'location' também seja um link encurtado ou
    redirecionado, ele será expandido; se não for possível expandir,
    a função retornará None.
    """
    final_url = expand_url(location)
    """
    Define o resultado como a URL final expandida, se obtida
    corretamente; caso contrário, usa a URL de redirecionamento
    ('location') como fallback
    """
    result = final_url or location

    # Armazena no cache todos redirecionamentos realizados
    cache[url] = result
    return result
    """
    Quando não há redirecionamento (status diferente dos códigos acima),
    ou há sucesso direto (ex: 200), ou erro (ex: 404, 500), define como None
    """

    expanded = None
    cache[url] = expanded
    return expanded

# Captura exceções específicas do requests
except RequestException as e:
    expanded = None
    cache[url] = expanded
    return expanded

# Captura outras exceções inesperadas
except Exception as e:
    expanded = None
    cache[url] = expanded
    return expanded

def expand_url_list(url_list):
    return [expand_url(url) for url in url_list]

```

```

[123]: # Cria coluna de URLs expandidas
df['urls_expand'] = None

```

```
[124]: # Ativar tqdm para pandas
tqdm.pandas()

# Cria a coluna urls_expand se não existir
df['urls_expand'] = df.get('urls_expand', pd.Series([None] * len(df)))

# Função de verificação
def needs_expansion(row):
    # Verifica se urls é uma lista válida
    if not isinstance(row['urls'], list) or len(row['urls']) == 0:
        return False

    # Verifica se urls_expand precisa de processamento
    if not isinstance(row['urls_expand'], list):
        return True # Não foi processado ainda

    # Verifica se há qualquer elemento None (expansão incompleta)
    return any(i is None for i in row['urls_expand'])

# Aplica a máscara
mask = df.apply(needs_expansion, axis=1)

# Processa apenas as linhas necessárias
df.loc[mask, 'urls_expand'] = df.loc[mask, 'urls'].
↳progress_apply(expand_url_list)
```

100%| | 17025/17025 [50:21<00:00, 5.64it/s]

Validando expansão:

```
[125]: # Calcula a quantidade de URLs expandidas
df['url_expand_count'] = df['urls_expand'].apply(
    lambda x: len([url for url in x if isinstance(url, str) and
                  url.strip()]) if isinstance(x, list) else 0
)
```

```
[126]: # Amostragem de validação
df[df['url_count'] > 0][
    ['urls', 'url_count', 'urls_expand', 'url_expand_count']].sample(10)
```

```
[126]:
```

	urls	url_count	\
45537	[https://t.co/pJFnAewRuy]	1	
62828	[https://t.co/h20ZI8Bn2h]	1	
46794	[https://t.co/Siprq3o0kj]	1	
18952	[https://t.co/1CsZYLTA0i]	1	
54962	[https://t.co/kZ5WgIjH6P]	1	
5014	[https://t.co/5kUN7k5Dr3]	1	

102278	[https://t.co/VdupnWxAs5]	1
12740	[https://t.co/KYk58CUADI]	1
45207	[https://t.co/RlEQXZBSN3]	1
95130	[https://t.co/9KJcn88PLk]	1

	urls_expand	url_expand_count
45537	[https://extra.globo.com/casos-de-policia/just...	1
62828	[https://twitter.com/kiIIbea/status/1585305204...	1
46794	[https://twitter.com/Marco90344165/status/1580...	1
18952	[https://twitter.com/MattheusMarque7/status/15...	1
54962	[https://twitter.com/brussel_ive/status/158304...	1
5014	[https://twitter.com/lamarinearaujo5/status/15...	1
102278	[https://twitter.com/Mauricio__Zap/status/1595...	1
12740	[https://twitter.com/jaehyunflr/status/1573465...	1
45207	[https://www.metropoles.com/colunas/guilherme-...	1
95130	[https://agenciabrasil.ebc.com.br/politica/not...	1

```
[127]: """
Filtrar tweets que contêm URLs, mas cuja quantidade de URLs expandidas é menor
que o total identificado. Isso pode indicar falhas na expansão; se houver muitos
casos assim, executar a função de expansão novamente.
"""
df[(df['url_count'] > 0) & (df['url_expand_count'] < df['url_count'])
][['urls', 'url_count', 'urls_expand', 'url_expand_count']]
```

```
[127]:          urls  url_count  urls_expand  url_expand_count
1043  [https://t.co/MZ1qJPcnuf]          1          [None]          0
```

```
[128]: print(expand_url('https://t.co/MZ1qJPcnuf'))
```

None

```
[129]: # Atualiza manualmente as URLs expandidas após verificação manual
df.at[1043, 'urls_expand'] = ['https://www.msn.com/pt-br/noticias/mundo/parte-2-
mais-pa%C3%ADses-que-usam-urna-eletr%C3%B4nica-em-elei%C3%A7%C3%B5es/ss-
AA11qCkf?ocid=ob-tw-ptbr-All']
df.at[1043, 'url_expand_count'] = 1
```

```
[130]: # Tratando URLs inválidas
url_regex = r'^((https?:/)?([a-zA-Z0-9-]+\.)+[a-zA-Z]{2,})(/[\w\.-]*)*/?$'

def validar_lista_urls(urls):
    if not isinstance(urls, list):
        return None

    urls_validas = []
    for url in urls:
        url_str = str(url).strip()
```

```

        if re.match(url_regex, url_str):
            urls_validas.append(url_str)

    return urls_validas if urls_validas else None

df['urls_expand_valid'] = df['urls_expand'].apply(validar_lista_urls)

```

```

[131]: # Calcula a quantidade de URLs validadas
df['url_expand_count_valid'] = df['urls_expand'].apply(
    lambda x: len([url for url in x if isinstance(url, str) and
                  url.strip()]) if isinstance(x, list) else 0
)

```

```

[132]: # Validando
df[df['url_count'] != df['url_expand_count']]
[['urls', 'url_count', 'urls_expand', 'url_expand_count']]

```

```

[132]: Empty DataFrame
Columns: [urls, url_count, urls_expand, url_expand_count]
Index: []

```

```

[ ]: # Qnd de URLs
df["url_count"].sum()

```

```

[ ]: np.int64(17713)

```

```

[ ]: # Qnd de URLs
df['url_expand_count'].sum()

```

```

[ ]: np.int64(17713)

```

B.5 Salvar Dataset Processado

```
[133]: # ===== Remoção das colunas e renomeação =====  
# Lista de colunas para remover  
colunas_para_remover = [  
    "raw_tweet_length",  
    "urls",  
    "url_count"  
]  
  
# Remove colunas (ignora caso não existam)  
df = df.drop(columns=colunas_para_remover, errors="ignore")  
  
# Mapeamento de renomeação das colunas  
mapeamento_renameacao = {  
    "urls_expand": "urls",  
    "url_expand_count": "url_count"  
}  
  
# Renomeia colunas  
df = df.rename(columns=mapeamento_renameacao)
```

```
[134]: new_order = [  
    # Identificação do Tweet (Conteúdo e Estrutura)  
    'tweet_id',  
    'raw_tweet',  
    'preprocessed_tweet',  
    'urls',  
    'hashtags',  
    'mentions',  
    'emojis',  
    'is_retweet',  
    'original_author',  
    'is_truncated',  
    'is_upper',  
    'tweet_length',  
    'has_keywords',  
  
    # Metadados do Tweet (Engajamento e Estatísticas)  
    'tweet_created_at',  
    'reply_to_tweet_id',  
    'reply_to_user_id',  
    'retweets',  
    'likes',  
    'engagement_raw',  
    'engagement_rate',  
    'engagement_index',
```

```

'url_count',
'hashtags_count',
'mentions_count',
'emoji_count',

# Informações do Usuário (Perfil e Conta)
'user_id',
'followers_count',
'user_created_at',
'is_verified',
'account_age_days',

# Metadados da Coleta (Contexto e Status)
'collection_date',
'is_withheld',
'electoral_phase'
]

# Reordenar o DataFrame
df = df[new_order]

```

[135]: df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 111217 entries, 0 to 111216
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   tweet_id              111217 non-null object
1   raw_tweet             111217 non-null object
2   preprocessed_tweet    111193 non-null object
3   urls                  17025 non-null  object
4   hashtags              111217 non-null object
5   mentions              111217 non-null object
6   emojis                111217 non-null object
7   is_retweet            111217 non-null int64
8   original_author       69620 non-null  object
9   is_truncated          111217 non-null int64
10  is_upper              111217 non-null int64
11  tweet_length          111217 non-null int64
12  has_keywords          111217 non-null int64
13  tweet_created_at      111217 non-null datetime64[ns]
14  reply_to_tweet_id     28331 non-null  object
15  reply_to_user_id      28535 non-null  object
16  retweets              111217 non-null Int64
17  likes                 111217 non-null int64
18  engagement_raw        111217 non-null Int64
19  engagement_rate       111217 non-null Float64

```

```
20 engagement_index    111217 non-null  Float64
21 url_count            111217 non-null  int64
22 hashtags_count       111217 non-null  int64
23 mentions_count       111217 non-null  int64
24 emoji_count          111217 non-null  int64
25 user_id              111217 non-null  object
26 followers_count      111217 non-null  int64
27 user_created_at      111217 non-null  datetime64[ns]
28 is_verified          111217 non-null  int64
29 account_age_days     111217 non-null  int64
30 collection_date      111217 non-null  datetime64[ns]
31 is_withheld          111217 non-null  int64
32 electoral_phase      111217 non-null  object
dtypes: Float64(2), Int64(2), datetime64[ns](3), int64(14), object(12)
memory usage: 28.4+ MB
```

```
[136]: df.to_csv("/content/drive/MyDrive/TCC/Data/TUEBR22.csv", index=False)
```